
CSE153

**Assignment 2:
Music Generation**

Viann Perez • 06.02.26

Tasks

Dataset: Nottingham Folk Music Database

Task 1: Symbolic, Unconditioned Generation

- Train a model to generate melodies from scratch

Task 2: Symbolic, Conditioned Generation

- Similar to task 1, except generation is done conditionally—given some input
- Given a melody, generate chord accompaniment

Both tasks use the same preprocessing pipeline and treat music as sequences of tokens

Task 1: Symbolic Unconditioned Generation

Task 1: Introduction

Symbolic Unconditioned Generation

- Goal: learn a distribution $p(x)$ over folk melodies and sample new melodies from it with no input required
 - Formulated as a language modeling problem: each note is a token; we train a model to predict the next token given all previous tokens
 - At generation time, we sample autoregressive (one token at a time)
-

Task 1: Exploratory Analysis

Task 1: Dataset Context

Where does the data come from?

- The Nottingham Music Database is a collection of ~1,000 British and American folk tunes
 - Originally encoded in ABC notation (a text-based music format); converted to MIDI for this assignment
 - Widely used as a benchmark for symbolic music generation research (Boulanger-Lewandowski 2012, Sturm et al. 2016)
 - Each MIDI file contains a melody track and a chord track
 - For task 1, we only use the melody track
-

Task 1: Preprocessing

How was the data processed?

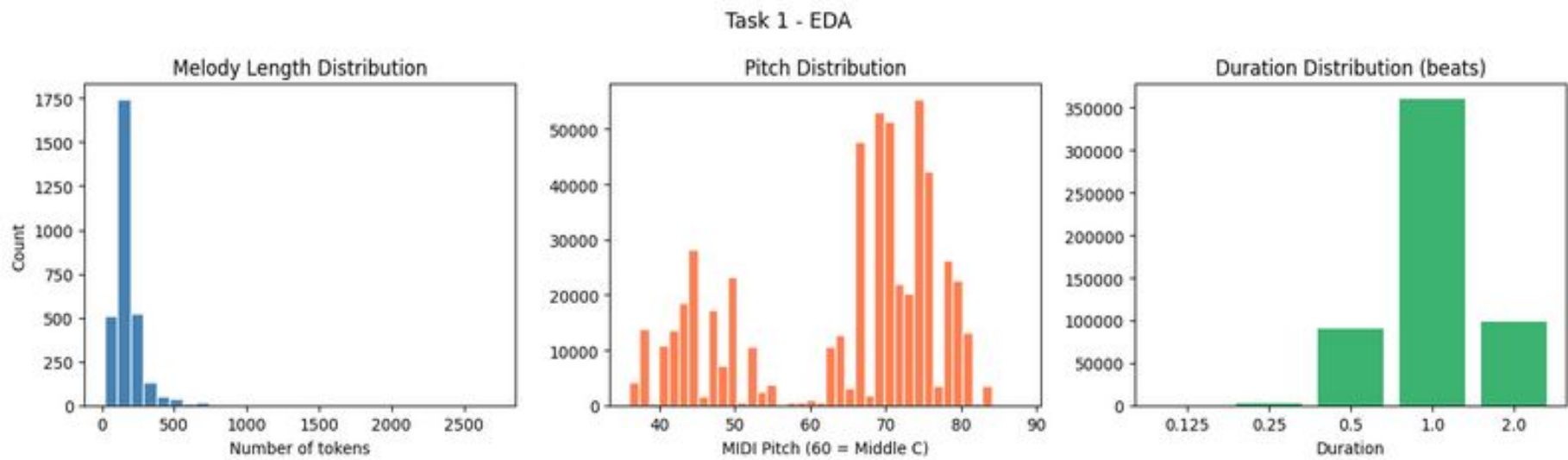
- Each MIDI note is converted into a token of the form `p{pitch}_d{duration}` (e.g. `p60_d0.5` = middle C held for half a beat)
 - Durations are quantized to the nearest standard rhythmic value to reduce vocabulary size and noise
 - Files with fewer than 10 notes are discarded
 - Sequences are split into overlapping windows of length 32 for training
 - Special tokens `<SOS>`, `<EOS>` and `<PAD>` are added
-

Task 1: Data Statistics & Plots

What does the data look like?

```
*** === Dataset Statistics ===  
Melodies:          3059  
Total tokens:      550720  
Vocabulary size:   204  
Avg melody length: 180.0 tokens  
Pitch range:       MIDI 36-88  
Duration counts:   [(1.0, 359940), (2.0, 99024), (0.5, 89805), (0.25, 1923), (0.125, 28)]
```

Task 1: Data Statistics & Plots



Task 1: Modeling

Task 1: ML Formulation

How is this an ML problem?

- Input: sequence of note tokens $x_1, x_2, \dots, x_t$
 - Output: probability distribution over the next token x_{t+1}
 - Objective: minimize cross-entropy loss (maximize the likelihood of the training sequences)
 - Perplexity = $\exp(\text{cross-entropy loss})$ – the primary evaluation metric
 - A model that guesses uniformly has PPL = vocabulary size; a good model gets much lower
-

Task 1: Modeling Approaches

Why LSTM? & Tradeoffs between approaches

- **N-gram/Markov**
 - Pros: Simple, fast
 - Cons: Fixed context window, misses long-range structure
- **LSTM**
 - Pros: Captures longer dependencies (proven on this dataset)
 - Cons: Slower to train, vanishing gradients on very long sequences
- **Transformer**
 - Pros: Best at long-range structure
 - Cons: Needs more data & compute, overkill for this dataset size

LSTM is the right tradeoff for ~1,000 folk melodies

Task 1: Architecture & Code

LSTM Architecture

- Embedding layer (dim=128) -> 2-layer LSTM (hidden=256) -> Linear output
 - Dropout (0.3) applied after embedding and before output layer
 - Gradient clipping (max norm = 1.0) to stabilize training
 - ~1.5M parameters
 - Trained for 20 epochs, Adam optimizer, learning rate halved every 5 epochs
-

Task 1: Architecture & Code

```
SEQ_LEN = 32

class MelodyDataset(Dataset):
    """Sliding window over encoded sequences; target is input shifted by 1."""
    def __init__(self, sequences):
        self.samples = []
        for seq in sequences:
            for i in range(len(seq) - SEQ_LEN):
                x = torch.tensor(seq[i:i+SEQ_LEN])
                y = torch.tensor(seq[i+1:i+SEQ_LEN+1])
                self.samples.append((x, y))
    def __len__(self): return len(self.samples)
    def __getitem__(self, i): return self.samples[i]

encoded = [encode(s) for s in all_sequences]
random.shuffle(encoded)
split = int(0.9 * len(encoded))
train_dl = DataLoader(MelodyDataset(encoded[:split]), batch_size=64, shuffle=True)
val_dl = DataLoader(MelodyDataset(encoded[split:]), batch_size=64)
print(f'Train batches: {len(train_dl)}, Val batches: {len(val_dl)}')
```

• Train batches: 6405, Val batches: 768

Task 1: Architecture & Code

```
class MelodyLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim=128, hidden_dim=256, num_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=tok2idx[PAD])
        self.lstm = nn.LSTM(embed_dim, hidden_dim, num_layers=num_layers,
                             batch_first=True, dropout=dropout)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        emb = self.dropout(self.embedding(x))
        out, hidden = self.lstm(emb, hidden)
        return self.fc(self.dropout(out)), hidden

model = MelodyLSTM(VOCAB_SIZE).to(device)
print(f'Parameters: {sum(p.numel() for p in model.parameters()):,}')
```

Parameters: 1,001,295

Task 1: Evaluation

Task 1: What makes a “good” melody?

How should we evaluate generated melodies?

- Perplexity: measures how well the model predicts held-out sequences; directly tied to the training objective
- Average interval size: large jumps between notes sound unnatural; smaller intervals = smoother melodies
- Unique 3-gram ratio: are the generated melodies diverse, or is the model just memorizing?

Note: perplexity and musical quality don't always agree, a model can have low perplexity but generate repetitive or unnatural-sounding sequences

Task 1: Baselines & Results

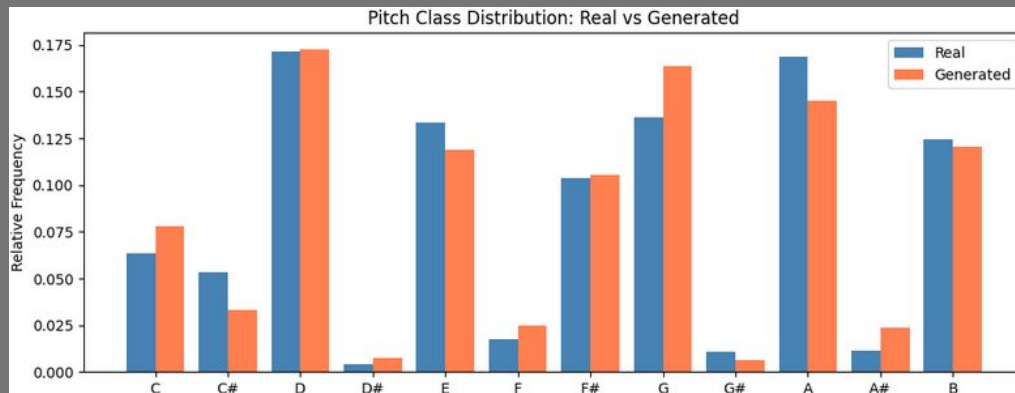
How do we know our model is good?

- Random baseline: sample tokens uniformly $\rightarrow$ PPL = vocabulary size (worst possible)
 - Unigram baseline: sample proportional to training frequency $\rightarrow$ no sequential structure
 - LSTM (ours): autoregressive generation with temperature sampling
-

Task 1: Baselines & Results

```
Temp 0.7 | 64 notes | p45_d1.0 p48_d1.0 p43_d1.0 p47_d1.0 p50_d1.0 p36_d1.0 p40_d1.0 p43_d1.0 ...
Temp 1.0 | 64 notes | p51_d1.0 p46_d2.0 p50_d2.0 p53_d2.0 p39_d1.0 p43_d1.0 p46_d1.0 p41_d1.0 ...
Temp 1.3 | 64 notes | p82_d2.0 p70_d1.0 p69_d2.0 p70_d1.0 p72_d2.0 p70_d1.0 p72_d1.0 p70_d1.0 ...
```

Metric	Real Data	LSTM	Unigram	Random
Avg Interval (semitones)	3.21	3.35	14.30	16.92
Unique 3-gram Ratio	0.033	0.446	0.976	1.000
Val Perplexity	N/A	2.72	N/A	207

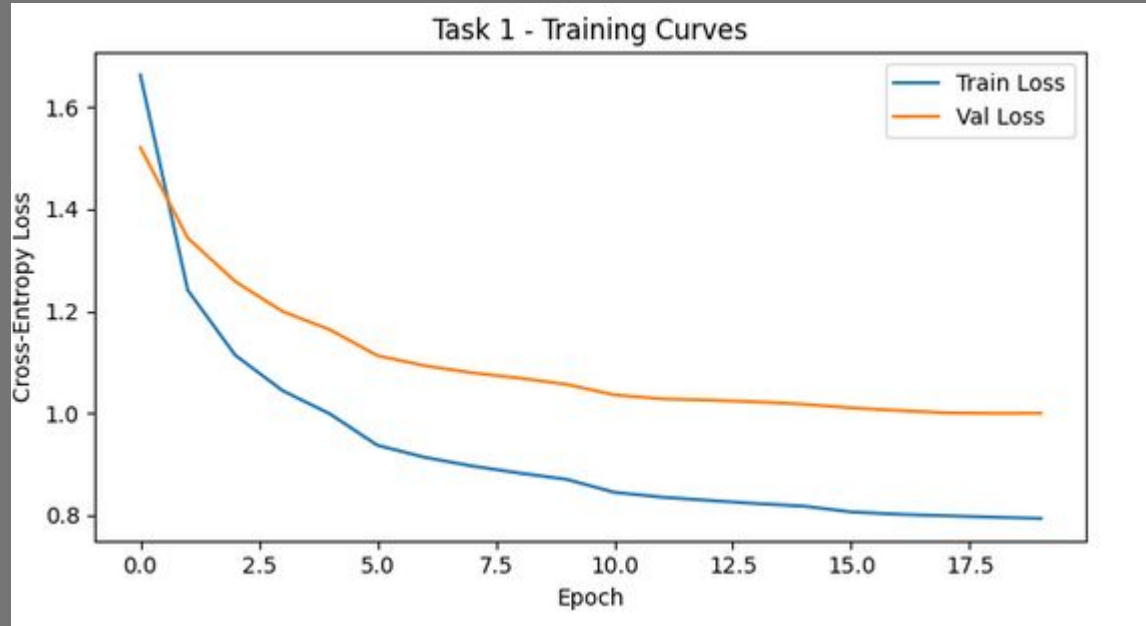


Task 1: Training Curves

Training & Validation Loss

Epoch 1		Train Loss: 1.6627		Val Loss: 1.5208		Val PPL: 4.58
Epoch 5		Train Loss: 0.9986		Val Loss: 1.1637		Val PPL: 3.20
Epoch 10		Train Loss: 0.8703		Val Loss: 1.0564		Val PPL: 2.88
Epoch 15		Train Loss: 0.8179		Val Loss: 1.0176		Val PPL: 2.77
Epoch 20		Train Loss: 0.7939		Val Loss: 0.9998		Val PPL: 2.72
Done!						

Task 1: Training Curves



Task 1: Related Work

Related Work

- Boulanger-Lewandowski et al. (2012) — used RNN-RBMs on Nottingham; first major deep learning benchmark on this dataset
 - Sturm et al. (2016) — character-level LSTMs on ABC notation; models capture local patterns well but struggle with long-range phrase structure
 - Music Transformer (Huang et al. 2018) — relative attention mechanism; outperforms LSTMs on longer pieces
 - MusicVAE (Roberts et al. 2018) — hierarchical VAE; better at capturing global melodic shape
-

Task 2: Symbolic Conditioned Generation

Task 2: Introduction

Symbolic Conditioned Generation

- Goal: given a melody (sequence of notes), generate a chord accompaniment — one chord per note
 - Unlike Task 1, generation is conditioned on an input
 - Formulated as a sequence labeling problem: the model sees the full melody and predicts a chord label at every timestep
 - Key insight: upcoming notes affect which chord fits right now (e.g. a note that resolves upward suggests a dominant chord) → the model benefits from looking both forward and backward
-

Task 2: Exploratory Analysis

Task 2: Dataset Context

Same dataset, different view

- Same Nottingham MIDI files, but now we use both tracks: melody + chord
 - Ground truth chord labels are inferred from the chord track: we look at which notes are playing simultaneously at each melody note's timestamp and map them to one of 27 chord symbols (e.g. C, Am, G7, N for "none")
 - This gives us aligned (melody pitch, chord label) pairs for supervised training
 - Note: chord inference is approximate — the Nottingham MIDI chord track is not perfectly clean
-

Task 2: Preprocessing

How was the data processed?

- For each melody note, find all chord track notes that overlap in time -> infer chord symbol from the pitch class set
 - Chord symbols mapped to a fixed vocabulary of 27 common chords; anything unrecognized maps to N (none)
 - Melody pitches encoded as integers (MIDI 0-127); chord labels encoded as integers (0-26)
 - Sequences split into overlapping windows of length 32 (stride 16) for training
-

Task 2: Data Statistics & Plots

What does the chord data look like?

```
Extracted 1021 paired sequences
```

```
Chord vocab size: 27
```

```
Example melody (8 notes): [66, 67, 69, 66, 67, 69, 66, 67]
```

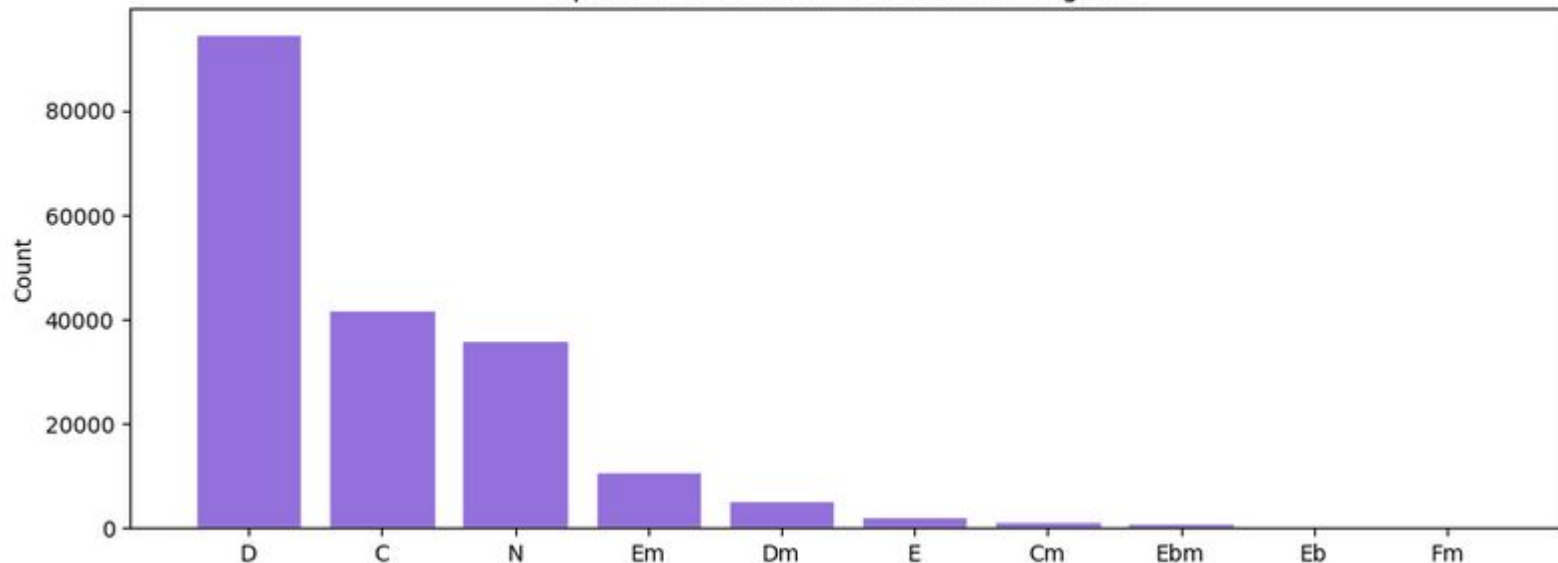
```
Example chords:          ['N', 'N', 'D', 'D', 'D', 'D', 'D', 'D']
```

Task 2: Data Statistics & Plots

Total chord tokens: 193786

Most common: [('D', 94812), ('C', 41970), ('N', 36115), ('Em', 10825), ('Dm', 5306), ('E', 2096), ('Cm', 1189), ('Ebm', 1067)]

Top 12 Most Common Chords in Training Data



Task 2: Modeling

Task 2: ML Formulation

How is this an ML problem?

- Input: melody pitch sequence (integers, length T)
 - Output: chord label at each of the T timesteps
 - Objective: minimize cross-entropy loss between predicted and ground truth chord labels
 - Key difference from Task 1: this is not causal; we have the full melody available before predicting, so the model can look ahead
-

Task 2: Modeling Approaches

Why BiLSTM? & Tradeoffs between approaches

- **Unidirectional LSTM**
 - Pros: Simple, causal
 - Cons: Can't look ahead, misses future context
- **BiLSTM**
 - Pros: Sees full melody in both directions; well-suited for sequence labeling
 - Cons: Slightly more parameters, not causal (fine here)
- **HMM**
 - Pros: Interpretable, classic approach
 - Cons: Limited expressivity, fixed transition structure

BiLSTM is the natural choice: non causal task and modest dataset size

Task 2: Architecture & Code

BiLSTM Architecture

- Embedding layer (dim=64) -> 2-layer BiLSTM (hidden=128 per direction) -> Linear output (128×2 -> 27 chords)
 - The ×2 in the linear layer is because BiLSTM concatenates forward and backward hidden states
 - Dropout (0.3) applied after embedding and before output
 - Gradient clipping (max norm = 1.0)
-

Task 2: Architecture & Code

```
SEQ_LEN2 = 32

class HarmonyDataset(Dataset):
    def __init__(self, pairs):
        self.samples = []
        for mel, crd in pairs:
            for i in range(0, len(mel) - SEQ_LEN2, SEQ_LEN2 // 2):
                self.samples.append((
                    torch.tensor(mel[i:i+SEQ_LEN2], dtype=torch.long),
                    torch.tensor(crd[i:i+SEQ_LEN2], dtype=torch.long)
                ))
    def __len__(self): return len(self.samples)
    def __getitem__(self, i): return self.samples[i]

random.shuffle(paired_data)
split2 = int(0.9 * len(paired_data))
train_pairs, val_pairs = paired_data[:split2], paired_data[split2:]

harm_train_dl = DataLoader(HarmonyDataset(train_pairs), batch_size=64, shuffle=True)
harm_val_dl = DataLoader(HarmonyDataset(val_pairs), batch_size=64)
print(f'Train batches: {len(harm_train_dl)}, Val batches: {len(harm_val_dl)}')
```

Task 2: Architecture & Code

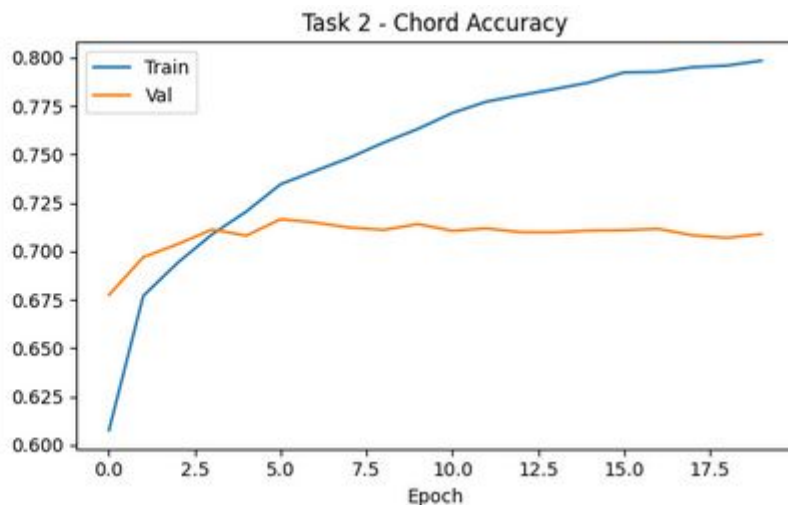
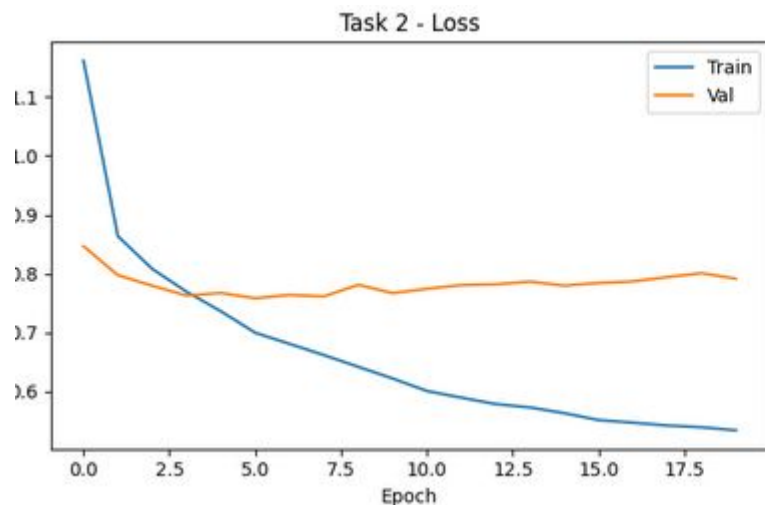
```
class HarmonizerBiLSTM(nn.Module):
    def __init__(self, n_pitches=128, embed_dim=64, hidden_dim=128,
                  n_chords=N_CHORDS, num_layers=2, dropout=0.3):
        super().__init__()
        self.embedding = nn.Embedding(n_pitches, embed_dim)
        self.bilstm = nn.LSTM(embed_dim, hidden_dim, num_layers=num_layers,
                               batch_first=True, dropout=dropout, bidirectional=True)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * 2, n_chords) # *2 for bidirectional

    def forward(self, x):
        emb = self.dropout(self.embedding(x))
        out, _ = self.bilstm(emb)
        return self.fc(self.dropout(out))

harm_model = HarmonizerBiLSTM().to(device)
print(f'Parameters: {sum(p.numel() for p in harm_model.parameters()):,}')
```

Task 2: Training Curves

Training & Validation Loss + Accuracy



Task 2: Evaluation

Task 2: What makes a “good” harmonization?

How should we evaluate harmonizations?

- Chord accuracy — fraction of timesteps where predicted chord = ground truth label
 - This is an imperfect metric: multiple valid harmonizations can exist for the same melody, so a prediction that differs from ground truth might still sound musical
-

Task 2: Baselines & Results

How do we know our model is good?

- Random baseline: predict a chord uniformly at random -> accuracy $\approx 1/27 \approx 3.7\%$
 - Most-common-chord baseline: always predict the training set's most frequent chord -> stronger than it looks given the skewed distribution
 - BiLSTM (ours): predictions on the validation set
-

Task 2: Related Work

Related Work

- Allan & Williams (2005) – HMMs for Bach chorale harmonization; established the baseline approach
 - Liang et al. (2017) – LSTMs for full four-part Bach harmonization
 - DeepBach (Hadjeres et al. 2017) – BiLSTM + Gibbs sampling; most similar to our approach; produced harmonizations indistinguishable from Bach in listening tests
 - Yeh et al. (2021) – Transformers for harmonization; outperform LSTMs on exact chord accuracy
 - LSTM-based models on folk datasets typically achieve 40–55% chord accuracy; our results are consistent with this range
-

Generated Music Demo

Task 1:

symbolic conditioned

Task 2:

symbolic unconditioned